HARI 1

Panduan lab hari 1

Building Modern Web Application

Praktik dimulai dengan menjalankan JavaScript di Node.js, lalu membangun server Express, menampilkan halaman EJS, menerapkan Bootstrap, dan menyajikan data karyawan melalui rute baru.

Urutan praktik

Lab	Topik praktik	Halaman
01	Eksplorasi Node.js dasar	2
02	Menyiapkan proyek Express	3
03	Implementasi EJS	4
04	Dasbor Bootstrap	6
05	Routing Express	8

LAB 01

Eksplorasi Node.js dasar

Tujuan

Peserta mampu menjalankan kode JavaScript sederhana di sisi server menggunakan Node.js (tanpa menggunakan peramban).

Langkah praktik

1. Buka Terminal (Linux/Mac) atau Command Prompt / PowerShell (Windows).
2. Ketikkan perintah `node` lalu tekan Enter. Anda akan masuk ke REPL (Read-Eval-Print-Loop) Node.js.
3. Di dalam REPL, ketikkan kode JavaScript sederhana:

```
let pesan = "Halo dari Server Node.js!";
console.log(pesan);
```

4. Anda akan melihat tulisan `Halo dari Server Node.js!` tercetak di layar terminal Anda.
5. Cobalah juga operasi matematika sederhana, misalnya:

```
let hasil = 10 * 5;
console.log("Hasil perkalian: " + hasil);
```

6. Keluar dari mode REPL dengan menekan `Ctrl + C` sebanyak dua kali.

Hasil pembelajaran

JavaScript dapat dijalankan langsung melalui Node.js di komputer atau server untuk melakukan komputasi dan tugas lainnya.

LAB 02

Menyiapkan proyek Express

Tujuan

Membuat server HTTP dasar yang dapat merespons teks sederhana menggunakan Express.js.

Langkah praktik

1. Buka VS Code. Buatlah sebuah folder baru di komputer Anda dengan nama web-app-saya lalu buka folder tersebut di VS Code.
2. Buka Terminal terintegrasi di VS Code (Terminal -> New Terminal).
3. Inisialisasi proyek Node.js:

```
npm init -y
```

Perintah ini akan menghasilkan berkas package.json.

4. Instal Express:

```
npm install express
```

5. Buat sebuah berkas baru bernama app.js di dalam folder proyek tersebut.
6. Ketikkan kode berikut ke dalam app.js:

```
const express = require('express');
const app = express();
const port = 3000;

// Routing dasar
app.get('/', (req, res) => {
  res.send('Hello World! Server Express sudah berjalan.');
});

// Menjalankan server
app.listen(port, () => {
  console.log(`Server berjalan di http://localhost:${port}`);
});
```

7. Jalankan server dengan perintah:

```
node app.js
```

8. Buka peramban dan akses alamat http://localhost:3000. Anda seharusnya melihat teks "Hello World! Server Express sudah berjalan."
9. Agar website dapat otomatis merestart aplikasi Node.js, anda bisa gunakan Nodemon dengan cara install

```
npm install --save-dev nodemon
```

10. Tambahkan nodemon pada scripts pada package.json:

```
"scripts": {
  "dev": "nodemon ./app.js",
  "test": "echo \"Error: no test specified\" && exit 1"
 },
```

Hasil pembelajaran

Hanya dengan beberapa baris kode, kita telah berhasil membuat web server lokal.

LAB 03

Implementasi EJS

Tujuan

Menampilkan berkas EJS yang merender variabel data dinamis dari server ke HTML. Peserta juga mampu membuat komponen navbar yang dapat digunakan ulang melalui partial.

Langkah praktik

1. Instal modul EJS melalui terminal:

```
npm install ejs
```

2. Pada berkas app.js, tambahkan konfigurasi EJS tepat di bawah const app = express();:

```
app.set('view engine', 'ejs');
```

3. Buat folder baru bernama views di direktori utama.
4. Di dalam folder views, buat berkas baru bernama index.ejs.
5. Ketikkan struktur HTML berikut di index.ejs:

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Aplikasi Node Saya</title>
</head>
<body>
  <h1>Selamat Datang, <%= namaPengguna %>!</h1>
  <p>Ini adalah aplikasi pertama menggunakan EJS.</p>
</body>
</html>
```

6. Ubah route / pada berkas app.js agar me-render EJS dan mengirimkan data:

```
app.get('/', (req, res) => {
  // Data ini seolah-olah berasal dari database
  const dataUser = { namaPengguna: 'Peserta Pelatihan' };
  res.render('index', dataUser);
});
```

7. Muat ulang peramban, pastikan tulisan yang muncul adalah "Selamat Datang, Peserta Pelatihan!".

Komponen reusable: navbar dengan partial EJS

Partial adalah potongan template EJS yang disimpan terpisah dan dipanggil dari beberapa halaman. Dengan cara ini, perubahan navbar cukup dilakukan pada satu berkas.

1 Buat folder views/partials, lalu buat berkas navbar.ejs di dalamnya. Isi views/partials/navbar.ejs dengan kode berikut (tanpa tag <html>, <head>, atau <body>):

```
<nav aria-label="Navigasi utama">
  <strong><%= namaAplikasi %></strong> |
  <a href="/">Beranda</a> |
  <a href="/tentang">Tentang</a>
</nav>
```

2 Pada views/index.ejs, sisipkan baris berikut tepat setelah tag pembuka <body>. Pertahankan isi halaman yang sudah ada:

```
<%- include('partials/navbar', { namaAplikasi: 'Aplikasi Node Saya' }) %>
```

Path partial dihitung relatif terhadap berkas pemanggil. Gunakan <%- include(...) %> agar HTML navbar dirender, bukan ditampilkan sebagai teks. Untuk nilai teks seperti namaAplikasi, tetap gunakan <%= ... %> agar karakter HTML di-escape.

3 Buat views/tentang.ejs untuk membuktikan bahwa navbar yang sama dapat digunakan pada halaman lain:

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Tentang Aplikasi</title>
</head>
<body>
  <%- include('partials/navbar', { namaAplikasi: 'Aplikasi Node Saya' }) %>
  <h1>Tentang Aplikasi</h1>
  <p>Halaman ini memakai navbar yang sama dengan Beranda.</p>
</body>
</html>
```

4 Di app.js, tambahkan route berikut sebelum app.listen(...). Jangan mengganti route / yang sudah ada:

```
app.get('/tentang', (req, res) => {
  res.render('tentang');
});
```

5 Simpan semua berkas. Jika tidak memakai nodemon, hentikan server dengan Ctrl + C lalu jalankan kembali node app.js. Buka http://localhost:3000/ dan klik Tentang, lalu Beranda. Pastikan navbar muncul pada kedua halaman.

6 Ubah label Beranda menjadi Halaman Utama hanya di views/partials/navbar.ejs. Muat ulang kedua halaman dan pastikan label berubah di keduanya tanpa menyunting index.ejs atau tentang.ejs.

Hasil pembelajaran

Satu partial navbar digunakan oleh dua halaman EJS. Struktur navigasi tidak perlu disalin, sementara data nama aplikasi dapat dikirim melalui parameter include. Navbar ini menggunakan HTML dasar dan tidak memerlukan Bootstrap.

LAB 04

Dasbor Bootstrap

Tujuan

Menerapkan Bootstrap CSS ke dalam berkas EJS untuk mempercantik tampilan dasar situs web tanpa menulis CSS sendiri.

Langkah praktik

1. Buka berkas views/index.ejs.
2. Di dalam tag <head>, tambahkan tautan Bootstrap CDN berikut:

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
```

3. Pertahankan pemanggilan partial navbar dari Lab 03 tepat setelah tag pembuka <body>, lalu ubah konten di bawah navbar dalam tag <body> menjadi seperti ini:

```
<div class="container mt-5">
  <div class="row justify-content-center">
    <div class="col-md-8">
      <div class="card shadow">
        <div class="card-header bg-primary text-white">
          <h4>Dashboard Sederhana</h4>
        </div>
        <div class="card-body">
          <h5 class="card-title">Halo, <%= namaPengguna %></h5>
          <p class="card-text">Bootstrap 5 telah berhasil dipasang di project Express.js!</p>
          <a href="#" class="btn btn-success">Mulai Belajar</a>
        </div>
      </div>
    </div>
  </div>
</div>
```

4. Jalankan ulang server dan cek kembali http://localhost:3000. Tampilan halaman akan berubah menjadi berbentuk *Card* rapi di tengah halaman.

Styling navbar reusable dari Lab 03

1 Tambahkan tautan Bootstrap CSS yang sama dari langkah sebelumnya ke dalam <head> pada views/tentang.ejs. Pastikan kedua halaman memiliki meta viewport berikut di dalam <head> agar tampilan mengikuti lebar perangkat:

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

2 Buka views/partials/navbar.ejs dan ganti isinya dengan kode berikut. Tetap gunakan parameter namaAplikasi dari Lab 03; jangan menyalin navbar ke setiap halaman:

```
<nav class="navbar bg-primary shadow-sm" data-bs-theme="dark"
   aria-label="Navigasi utama">
  <div class="container flex-wrap gap-2">
    <a class="navbar-brand" href="/"><%= namaAplikasi %></a>
    <div class="navbar-nav flex-row gap-3 ms-auto">
      <a class="nav-link" href="/">Beranda</a>
      <a class="nav-link" href="/tentang">Tentang</a>
    </div>
  </div>
</nav>
```

Kelas navbar mengatur navigasi, bg-primary memberi latar biru, dan data-bs-theme="dark" menyesuaikan warna tautan pada latar gelap. container menyelaraskan lebar navbar dengan konten dasbor; flex-wrap dan gap-2 memberi ruang saat layar menyempit. Menu selalu terlihat, sehingga contoh ini tidak memerlukan Bootstrap JavaScript atau tombol collapse.

3 Pastikan index.ejs dan tentang.ejs tetap memanggil partial berikut tepat setelah <body>. Pada index.ejs, letakkan konten dasbor di bawah pemanggilan ini, bukan di dalam partial:

```
<%- include('partials/navbar', { namaAplikasi: 'Aplikasi Node Saya' }) %>
```

4 Simpan berkas dan muat ulang halaman Beranda serta Tentang. Pastikan navbar berlatar biru tampil konsisten, kedua tautan berfungsi, dan menu tetap terbaca saat lebar peramban diperkecil. Ubah bg-primary menjadi bg-success hanya di partial, lalu muat ulang kedua halaman untuk membuktikan bahwa styling berubah bersama.

Hasil: navbar dari Lab 03 kini memakai Bootstrap tanpa menghilangkan sifat reusable. Perubahan struktur dan kelas navbar tetap dilakukan pada satu berkas, sedangkan Bootstrap CSS dimuat oleh setiap halaman pemanggil.

LAB 05

Routing Express

Tujuan

Membuat rute baru menggunakan method GET, mengirim array data ke View, lalu menampilkannya sebagai baris-baris tabel dengan iterasi (loop) di EJS.

Langkah praktik

1. Buka app.js dan tambahkan sebuah routing baru untuk URL /karyawan:

```
app.get('/karyawan', (req, res) => {
  const dataKaryawan = [
    { id: 1, nama: 'Budi Santoso', divisi: 'IT' },
    { id: 2, nama: 'Siti Aminah', divisi: 'Finance' },
    { id: 3, nama: 'Agus Setiawan', divisi: 'Marketing' }
  ];
  res.render('karyawan', { listKaryawan: dataKaryawan });
});
```

2. Buat berkas baru bernama karyawan.ejs di dalam folder views.
3. Masukkan kode berikut ke dalam karyawan.ejs:

```
<!DOCTYPE html>
<html lang="id">
<head>
  <title>Data Karyawan</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body class="container mt-5">
  <h2>Daftar Karyawan</h2>
  <table class="table table-bordered table-striped mt-3">
    <thead class="table-dark">
      <tr>
        <th>ID</th>
        <th>Nama Karyawan</th>
        <th>Divisi</th>
      </tr>
    </thead>
    <tbody>
      <% listKaryawan.forEach(function(karyawan) { %>
      <tr>
        <td><%= karyawan.id %></td>
        <td><%= karyawan.nama %></td>
        <td><%= karyawan.divisi %></td>
      </tr>
      <% }); %>
    </tbody>
  </table>
  <a href="/" class="btn btn-secondary">Kembali</a>
</body>
</html>
```

4. Jalankan ulang server Anda.
5. Akses http://localhost:3000/karyawan di peramban. Anda akan melihat sebuah tabel Bootstrap yang baris datanya dihasilkan berdasarkan iterasi JavaScript di dalam EJS (<% ... %>).

Latihan tambahan: menampilkan daftar produk

Latihan ini memakai pola yang sama dengan daftar karyawan: membuat route GET, mengirim array ke EJS, lalu menampilkannya sebagai tabel. Data tetap ditulis langsung di app.js; tidak perlu database atau modul tambahan.

1 Di app.js, tambahkan route /produk berikut sebelum app.listen(...). Pertahankan route /karyawan yang sudah ada:

```
app.get('/produk', (req, res) => {
  const dataProduk = [
    { id: 1, nama: 'Buku Tulis', kategori: 'Alat Tulis' },
    { id: 2, nama: 'Pulpen', kategori: 'Alat Tulis' },
    { id: 3, nama: 'Tas Sekolah', kategori: 'Perlengkapan' }
  ];
  res.render('produk', { listProduk: dataProduk });
});
```

2 Salin views/karyawan.ejs menjadi views/produk.ejs agar struktur HTML dan tabel Bootstrap tidak perlu diketik ulang. Jangan mengubah berkas karyawan.ejs.

3 Pada produk.ejs, ubah judul Data Karyawan menjadi Data Produk, judul Daftar Karyawan menjadi Daftar Produk, serta header tabel Nama Karyawan menjadi Nama Produk dan Divisi menjadi Kategori. Header ID tetap sama.

4 Masih di produk.ejs, ganti seluruh bagian perulangan di dalam <tbody> dengan kode berikut. Tag <tbody> dan </tbody> tetap dipertahankan:

```
<% listProduk.forEach(function(produk) { %>
<tr>
  <td><%= produk.id %></td>
  <td><%= produk.nama %></td>
  <td><%= produk.kategori %></td>
</tr>
<% }); %>
```

5 Simpan semua berkas dan jalankan ulang server jika tidak memakai nodemon. Buka http://localhost:3000/produk. Pastikan tabel menampilkan tiga produk dengan kolom ID, Nama Produk, dan Kategori. Cek juga /karyawan untuk memastikan halaman sebelumnya tetap berfungsi.

6 Tambahkan satu produk lagi pada array dataProduk di app.js. Beri koma setelah objek produk ketiga, lalu tambahkan objek berikut sebelum tanda];:

```
{ id: 4, nama: 'Penggaris', kategori: 'Alat Tulis' }
```

Simpan app.js, jalankan ulang server bila diperlukan, lalu muat ulang /produk. Hasil yang diharapkan: jumlah baris data bertambah dari tiga menjadi empat tanpa menambahkan baris HTML secara manual.